

ROLE

You are a Senior Full Stack Architect, AI Engineer, Backend Engineer, Frontend Engineer, and DevOps Engineer working on a production-grade SaaS application.

Your task is to design and implement a complete Construction Progress Monitoring System.

The system must follow modern software engineering practices, clean architecture principles, scalable backend design, and production-ready code standards.

PROJECT OVERVIEW

Build a web application that allows users to:

1. Upload construction site videos and images.
2. Select and compare different timestamps or date ranges.
3. Analyze construction progress using AI.
4. Detect construction-related objects.
5. Calculate progress metrics.
6. Generate visual reports.
7. Display analytics through a modern dashboard.

The primary goal is to monitor construction progress over time and identify changes between different periods.

REQUIRED ARCHITECTURE

Frontend:

- Next.js (App Router)
- TypeScript
- Tailwind CSS
- shadcn/ui
- Recharts

Backend:

- Go
- Gin Framework
- PostgreSQL
- GORM

AI Service:

- Python

- FastAPI
- OpenCV
- YOLO
- FFmpeg

Storage:

- Local Storage (MVP)
- MinIO-ready architecture

Queue:

- Redis
- Background Workers

Deployment:

- Docker
- Docker Compose

SYSTEM ARCHITECTURE

Frontend (Next.js) | | v Backend API (Go + Gin) | | +-----+ | | v v PostgreSQL Redis Queue | | v
Python AI Service | | YOLO/OpenCV | | Storage

DEVELOPMENT PHASES

PHASE 1

Project Initialization

Tasks:

- Setup monorepo structure
- Configure Docker
- Configure Docker Compose
- Create frontend project
- Create Go backend project
- Create Python AI service
- Setup PostgreSQL
- Setup Redis
- Setup environment management

Folder Structure:

/frontend /backend /ai-service /docker /storage /docs

Deliverables:

- All services start successfully
 - Docker Compose works
 - Frontend can communicate with backend
-

PHASE 2

Authentication & Project Management

Features:

- Login
- Register
- JWT Authentication
- User Profile

Construction Projects:

- Create Project
- Update Project
- Delete Project
- View Project

Database Entities:

users projects

Deliverables:

- Authentication complete
 - Project CRUD complete
-

PHASE 3

Media Upload System

Features:

- Upload Images
- Upload Videos
- Drag & Drop Upload
- Progress Indicator

Backend:

- File Storage
- Metadata Storage

Database:

media_files

Fields:

id project_id file_name file_type file_path uploaded_at

Deliverables:

- Media uploaded successfully
- Metadata saved

PHASE 4

Video Processing Pipeline

Python Service Responsibilities:

- Extract frames
- Generate thumbnails
- Generate timeline snapshots

FFmpeg Processing:

- Extract frame every N seconds
- Save frames

Database:

frames

Fields:

id media_id timestamp frame_path

Deliverables:

- Frames extracted automatically
- Timeline generated

PHASE 5

AI Object Detection

Integrate YOLO.

Detect:

- Workers
- Helmets
- Trucks
- Cranes
- Excavators
- Scaffolding
- Pillars
- Walls
- Construction Equipment

Store results:

detections

Fields:

id frame_id object_type confidence bounding_box

Deliverables:

- AI detection working
- Detection results stored

PHASE 6

Construction Progress Analysis

Goal:

Compare two dates, images, or videos.

Analysis:

- Object count differences
- Area changes
- Structural growth
- New object appearance
- Removed object detection

Generate metrics:

- Progress percentage
- Growth percentage
- Change score

Deliverables:

- Progress engine working
-

PHASE 7

Dashboard

Create a modern SaaS dashboard.

Pages:

Dashboard Projects Media Analysis Reports Settings

Widgets:

- Total Projects
- Uploaded Videos
- Progress Percentage
- AI Detection Count

Charts:

- Progress Over Time
- Detection Statistics
- Activity Trends

Deliverables:

- Dashboard complete
-

PHASE 8

Comparison Interface

Features:

- Before / After Viewer
- Split Screen Comparison
- Timeline Comparison
- AI Overlay Viewer

User can:

- Select date A
- Select date B
- Compare

Deliverables:

- Comparison UI complete
-

PHASE 9

AI Summary Generator

Python Service generates summaries.

Example:

"North building structure increased by 18% compared to the previous inspection."

Provide:

- Daily summary
- Weekly summary
- Monthly summary

Store reports:

progress_reports

Fields:

id project_id summary progress_percentage generated_at

Deliverables:

- AI reports generated automatically
-

PHASE 10

Background Processing

Implement job queue.

Workflow:

Upload Video → Create Job → Push to Redis → Python Worker → Frame Extraction → AI Detection → Save Results

Requirements:

- Retry mechanism
- Failed jobs

- Job status tracking

Deliverables:

- Async processing complete
-

API REQUIREMENTS

Backend APIs must include:

Authentication APIs

Project APIs

Media APIs

Analysis APIs

Report APIs

Job Status APIs

All APIs must:

- Use REST conventions
 - Use DTO validation
 - Use Swagger documentation
 - Return standardized responses
-

FRONTEND REQUIREMENTS

Use:

- Server Components where appropriate
- React Query
- Zustand
- TypeScript strict mode

UI Requirements:

- Responsive
- Dark Mode
- Modern SaaS design
- Loading states
- Error handling
- Empty states

CODE QUALITY REQUIREMENTS

- Clean Architecture
- SOLID Principles
- Repository Pattern
- Service Layer
- Dependency Injection
- Unit Tests
- Integration Tests
- Type Safety
- Logging
- Error Handling

MVP PRIORITY

Implement only these first:

1. Authentication
2. Project Management
3. Upload Video/Image
4. Frame Extraction
5. YOLO Detection
6. Progress Comparison
7. Dashboard

Everything else should be treated as secondary features.

Generate the entire project incrementally phase by phase and do not skip architecture planning before implementation.